Introduction :

- Federated Learning (FL) is ML paradigm introduced for collaborative model training
- The data owned by each client is private, and not shared with other clients or the server. Therefore, each client participates in FL by sharing only the model updates
- Why use fl : Privacy protection , data security ,
- If they ask how is it different from distributed optimization :
  (First, the data across clients is non-IID, meaning that a single client's local dataset may not accurately represent the overall population distribution. Second, the data is unbalanced, as the amount of private data available to each client can vary significantly.
)
- full client participation can lead to increased communication overhead, longer training time Moreover, clients with poor quality or highly biased local data may negatively impact the performance
- So we need a so we need a client selection strategy

TASK 1 :

Dataset used : MNIST (60,000 images of handwritten digits 0–9 , testing - 10000) - We want the model to identify handwritten digits
No of clients - 50 , no of clients for selection : 10
The data distribution is non iid and we used extreme 1-class-non iid principal to do that. (if they ask about this : sort based on labels and make 100 groups , shuffle those groups and assign 2 groups to each client)

First strategy which we have implemented is :
DivFL (diversity aware FL) :
In DivFL we try to choose clients whose model updates are most diverse among those 50 clients

1. Each client computes a local model update: $\Delta_i = w_i - w_g$
2. Initialize selected set as empty $S = \emptyset$ and coverage as $m(u) = \infty$ for all clients u.
3. $m(u)$ represents the minimum distance of client u to any selected client: $m(u) = \min_{s \in S} d(u, s)$.
4. Flatten updates and measure pairwise diversity using Euclidean distance: $d(i, j) = \|\Delta_i - \Delta_j\|_2$
5. For a candidate client i, update coverage as $m'(u) = \min(m(u), d(u, i))$.
6. Select client greedily maximizing gain: $\text{Gain}(i) = \sum_u m(u) - \sum_u m'(u)$.

2nd Strategy :

Shapely value based method : It measures how much each client's update contributes to improving model performance .

1. Generate R random permutations of all n clients and evaluate each client's marginal contribution (increase in validation accuracy after adding that client in a permutation).
2. Shapley value of client k is computed as: $SV_k = \frac{1}{R} \sum_{\pi=1}^{R} [v(S_{before} \cup \{k\}) - v(S_{before})]$
3. Apply EMA smoothing to combine past and current contributions, giving stable long-term relevance scores: $\phi_k \leftarrow 0.75\phi_k + 0.25SV_k(t)$
4. Convert EMA relevance scores into softmax probabilities and sample k clients, where higher ϕ_k means higher probability of being selected: $P(k) = \frac{e^{\phi_k}}{\sum_j e^{\phi_j}}$

We trained CNN , Logistic Regression , SVM and DNN models under both the selection strategies and below is the result :

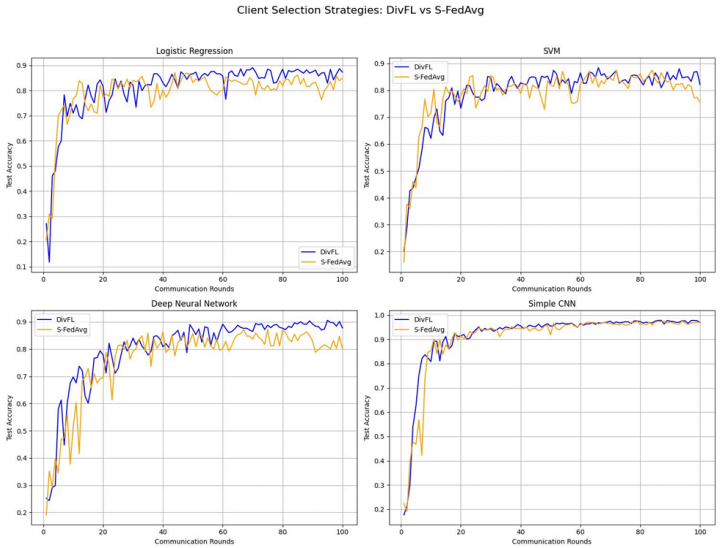


Figure 1: Task 1: DivFL (blue) vs S-FedAvg (orange) across 4 models, 100 rounds, stable environment.

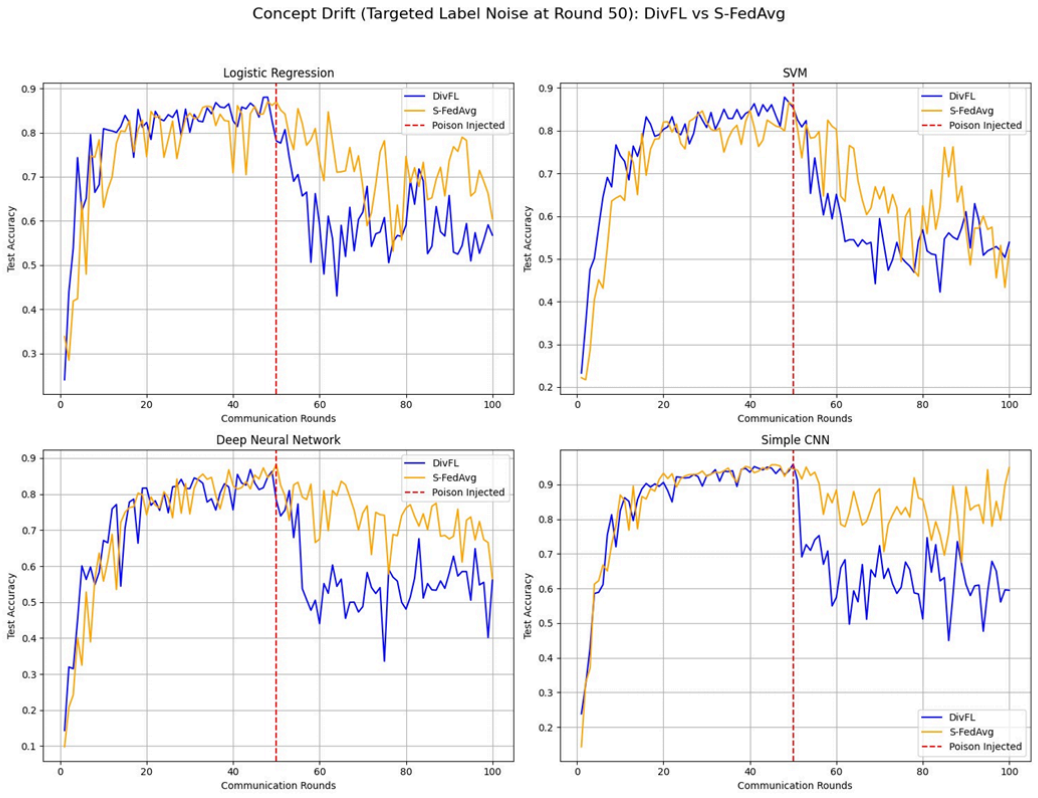
Why divFL performs better in initial rounds :
because it immediately selects clients with diverse updates, so the model sees many different digit patterns/classes from the very first round.

S-FedAvg starts with nearly equal relevance scores for all clients, so early client selection is almost random. It needs several rounds to estimate Shapley values and identify which clients are actually useful.

TASK 2 :
What is the underlying data distribution is non-stationary and constantly evolves. This phenomenon is commonly referred to as concept drift.

In our experiment: at round 50, clients 0–14 (out of 50) get their labels secretly shifted: ynew = (yoriginal + 1) mod 10
These 15 clients now consistently send wrong updates that push the model in the opposite direction.

This is the result :



DivFL :

Why DivFL Collapses When 15 clients get poisoned, their gradients change drastically — they now point in completely wrong directions. From DivFL's perspective, these are now the most diverse gradients in the entire pool!

CNN accuracy: dropped from 95.87% at round 50 to 57.46% by round 60. Never recovered. Final: 59.49%.

SFedAvg :
When a poisoned client is selected, its update reduces validation accuracy, giving it a negative Shapley value.
The client's EMA relevance score gradually decreases.
Lower EMA scores reduce the client's softmax selection probability, so poisoned clients are selected less frequently over time.
Recovery is slow because EMA retains past positive history (alpha=0.75) , so bad clients are forgotten gradually rather than immediately. (it takes 10-15 rounds to completely suppress a bad client)

• DivFL has a fundamental design flaw for adversarial settings — diversity at traction becomes a vulnerability
• S-FedAvg is better but has fixed memory — it can't speed up forgetting when things go bad quickly
• We need: Shapley-based reward + adaptive forgetting speed + explicit client isolation This is exactly what FedCADS-UCB is built to solve .

TASK 3 :

FedCADS-UCB stands for: Federated CUSUM-Adaptive Discounted Shapley UCB.

Working :

1. Memory Statistics ($\tilde{R}_k$ and $\tilde{N}_k$)

$\tilde{R}_k$: A client's discounted sum of past Shapley rewards (their "reputation").

$\tilde{N}_k$: A client's discounted selection count (their "attendance").

Equations:

$$\tilde{R}_k(t+1) = \gamma \cdot \tilde{R}_k(t) + SV_k(t) \cdot 1[k \in S_t]$$

$$\tilde{N}_k(t+1) = \gamma \cdot \tilde{N}_k(t) + 1[k \in S_t]$$

Significance: γ controls forgetting speed. High γ (0.95) means a long memory, while low γ (0.30) forces rapid forgetting.

2. UCB_k (Exploration vs. Exploitation)

Formula: $UCB_k = \hat{\mu}_k + \sqrt{\frac{\ln(t+1)}{\tilde{N}_k}}$.

Exploitation: The $\hat{\mu}_k$ term leverages clients known to have high average rewards.

Exploration: The square root term acts as a bonus that inflates when a client's selection count ($\tilde{N}_k$) is low, forcing the algorithm to try untested clients.

3. CUSUM (Drift Detection)

Measures accuracy drops: $g_t = acc_{t-2} - acc_{t-1}$.

Accumulates sustained drops (ignoring minor noise via a 0.01 slack): $C_t = \max(0, C_{t-1} + g_t - 0.01)$.

If the tally $C_t > 0.05$, an attack/drift is declared.

4. Adaptive γ

CUSUM directly drives the memory factor: $\gamma(t) = 0.95 - 0.65 \times \min(C_t/0.05, 1)$.

When stable, $\gamma = 0.95$. If drift triggers the CUSUM alarm, γ instantly drops to 0.30 to quickly forget the clients' now-irrelevant old reputations.

5. Fairness Debt & Final Score

Debt (D_k): A virtual queue that tracks how unfairly a client has been treated. It increases if a client is ignored and decreases when they are picked.

$$D_k(t) = \max(0, D_k(t-1) + c_k - b_k(t-1))$$

(Where $c_k = 0.01$ target, and $b_k = 1$ if selected last round, else 0)

Final Score: Combines performance and fairness: $Score_k(t) = 0.9 \times UCB_k(t) + 0.1 \times D_k(t)$.

The algorithm ranks all active clients and selects the 10 with the highest final scores for local training.

7. Quarantine & Two Global Models

Quarantine: If a selected client returns a harmful update ($SV_k < -0.01$) for two consecutive rounds, they are removed from the active pool with an exponential backoff timeout.

Two Models: Quarantined clients might represent a genuinely new data concept rather than pure noise. To preserve this, h_{global} trains on clean updates, while h_{drift} trains exclusively on quarantined updates.

8. Why FedCADS-UCB Beats the Competition

S-FedAvg: Uses fixed memory ($\alpha = 0.75$) and adapts too slowly when poisoned.

CUCB-SW: Uses a fixed sliding window. This is brittle because you cannot predict the optimal window size before a drift occurs.

CUCB-BoB: Runs multiple window sizes simultaneously, which adds massive complexity but still relies on guessing rather than measuring the drift.

FedCADS-UCB: Directly links the physical drift measurement (CUSUM) to memory adaptation in real-time, forgetting exactly as fast as it needs to.

Result :

Task 3 Results — Reading Every Detail

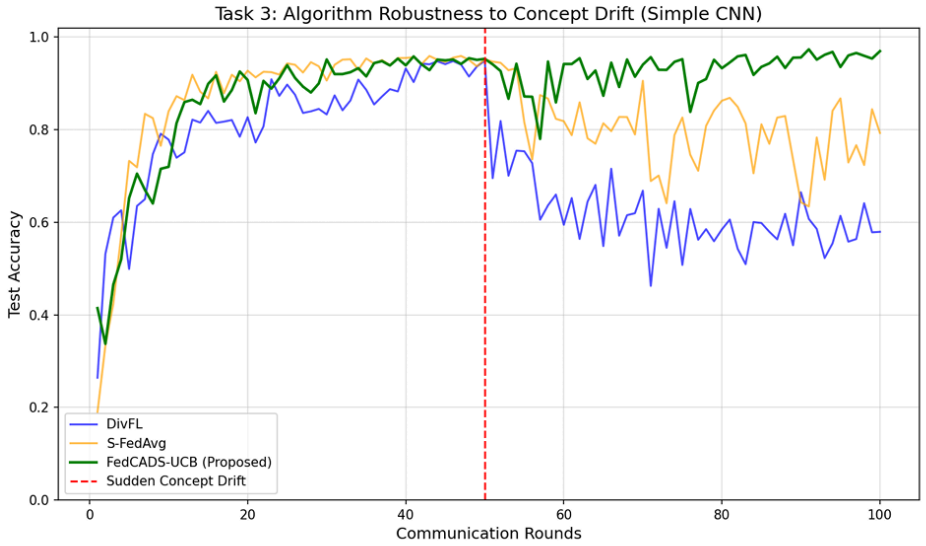


Figure 3: Task 3: FedCADS-UCB (green) vs DivFL (blue) and S-FedAvg (orange). Drift injected at round 50.

Even in phase 3, the green line oscillates between 83% and 97%. This is because: 1. Some quarantined clients occasionally return from shorter timeouts and immedi ately cause 1–2 rounds of damage before being re-quarantined. 2. Monte Carlo Shapley (R = 5) has inherent noise — sometimes a good client gets a slightly negative SV by chance. 3. The fairness debt occasionally forces selection of a client that turns out to be not very useful in that round. Despite the noise, the trend is stable and upward, which is what matters

END——————————————